3D WEBSOFT INC. FOUNDATION

FUTURE TECH SCHOOL PROGRAM

Faculty Name :

Course Duration : 5 Months

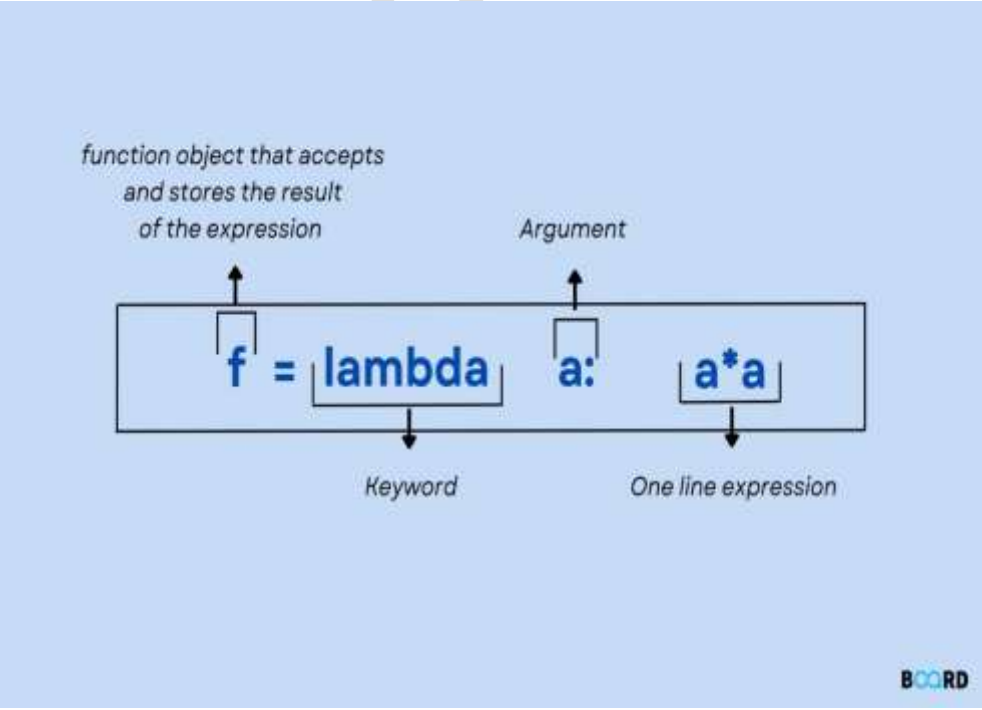
Classes : Offline & Online

Daily Teaching : 2 Hours

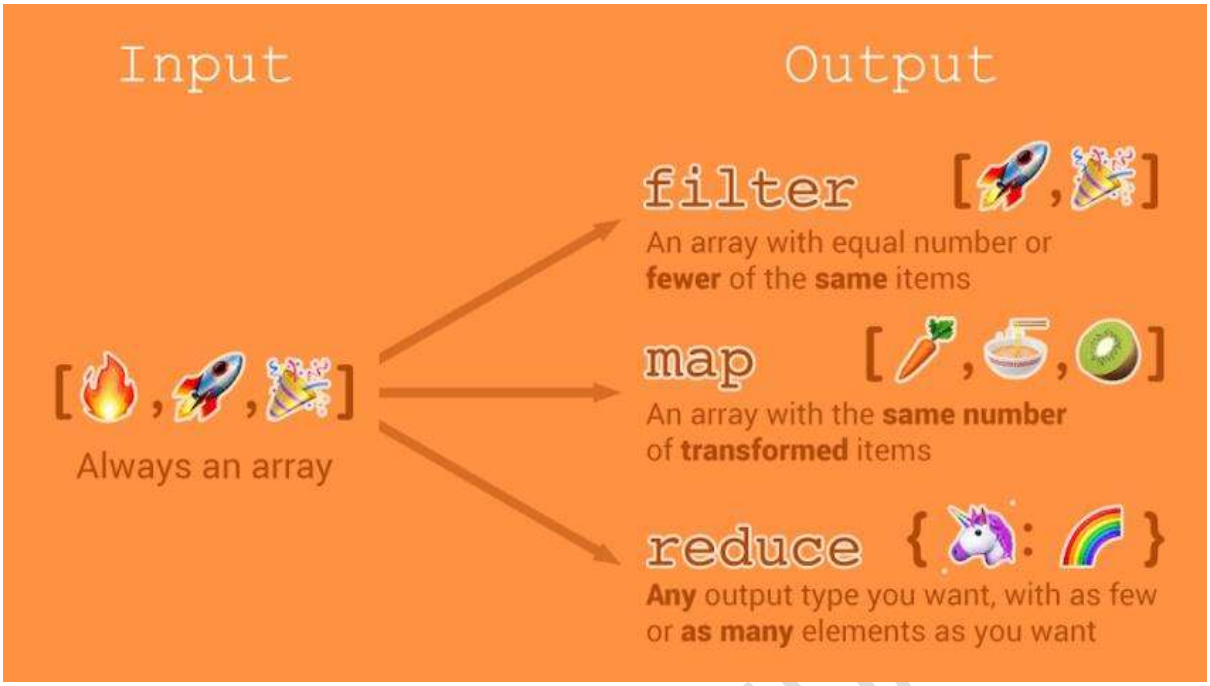
Total Classes : 100

Level 4 – AI Professional Foundation(11th–12th Standard)

Class 11: Lambda Functions & Small Data-Processing Programs



3D WEBSOFT INC. FOUNDATION



1. Learning Objectives

By the end of this lesson, students will be able to:

- Understand what a **Lambda Function** is.
- Write simple lambda functions.
- Use lambda functions with map(), filter(), and sorted().
- Process lists of numbers and student data.
- Create small real-world data-processing programs.
- Understand when to use a lambda function instead of a normal function.

3DWEBSOFT FOUND

2. What is a Lambda Function?

A **lambda function** is a small, anonymous function in Python.

Anonymous means that the function does not have a regular name.

A normal function:

```
def square(x):
    return x * x
```

The same operation using lambda:

```
square = lambda x: x * x
```

Basic Syntax

lambda arguments: expression

Example:

```
lambda x: x * 2
```

Here:

- lambda → keyword used to create the function
- x → argument/input
- : → separates input and expression
- x * 2 → result

3. Simple Example

```
double = lambda x: x * 2

print(double(5))
```

Output

10

How it works

5

3DWEBSOFT FOUNDATION

↓

lambda x: x * 2

↓

5 × 2

↓

10

4. Lambda vs Normal Function

Normal Function

```
def add(a, b):
    return a + b

print(add(10, 20))
```

Lambda Function

```
add = lambda a, b: a + b

print(add(10, 20))
```

Output

30

Normal Function	**Lambda Function**
Uses def	Uses lambda
Usually has a name	Usually anonymous

3DWEBSOFT FOUNDATION

Normal Function	Lambda Function
Can contain multiple statements	Contains one expression
Suitable for larger functions	Suitable for small operations

5. Lambda with One Argument

```
cube = lambda x: x ** 3


print(cube(3))
```

Output:

27

Another example:

```
positive = lambda x: x > 0


print(positive(10))
```

Output:

True

6. Lambda with Multiple Arguments

A lambda can accept multiple arguments.

```
multiply = lambda a, b: a * b


print(multiply(5, 4))
```

3DWEBSOFT FOUNDATION

Output:

20

Another example:

```
maximum = lambda a, b: a if a > b else b


print(maximum(15, 20))
```

Output:

20

7. Lambda with Lists

Suppose we have:

```
numbers = [1, 2, 3, 4, 5]
```

We can use lambda to perform an operation on each number.

```
square = lambda x: x * x


for number in numbers:
    print(square(number))
```

Output:

1

4

9

16

25

3DWEBSOFT FOUNDATION

8. map() + Lambda

map() is used when we want to perform an operation on **every item** in a collection.

Example

numbers = [1, 2, 3, 4, 5]

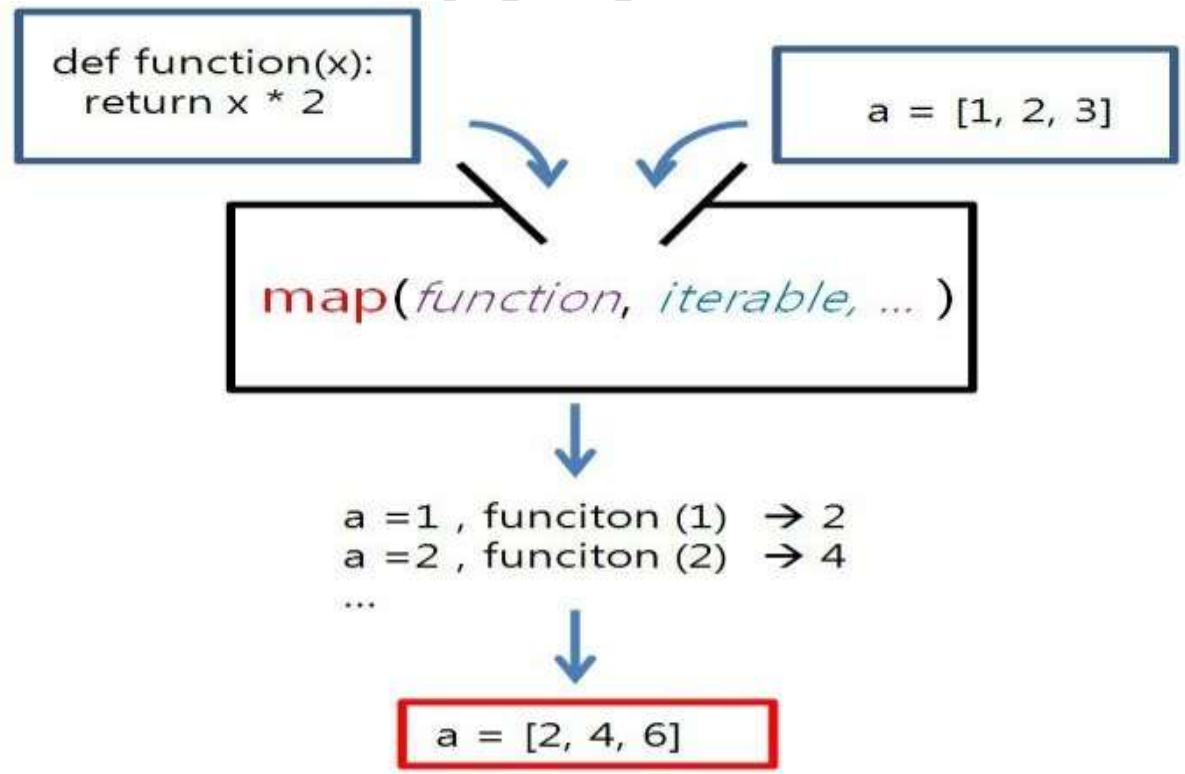
squares = list(map(lambda x: x * x, numbers))

print(squares)

Output:

[1, 4, 9, 16, 25]

Visual idea



FOUNDATION

[1, 2, 3, 4, 5]

↓

lambda x:x*x

↓

[1, 4, 9, 16, 25]

9. filter() + Lambda

filter() is used to **select items that satisfy a condition**.

Example: Find Even Numbers

```
numbers = [1, 2, 3, 4, 5, 6]

even_numbers = list(filter(lambda x: x % 2 == 0, numbers))

print(even_numbers)
```

Output:

[2, 4, 6]

How it works

1 → False

2 → True

3 → False

4 → True

5 → False

6 → True

3DWEBSOFT FOUNDATION

Result → [2, 4, 6]

10. sorted() + Lambda

Lambda functions are commonly used with sorted().

Suppose:

```
students = [
    ("Ali", 85),
    ("Sara", 92),
    ("John", 78)
]
```

Sort students according to marks:

```
students = [
    ("Ali", 85),
    ("Sara", 92),
    ("John", 78)
]

result = sorted(students, key=lambda student: student[1])

print(result)
```

Output:

```
[('John', 78), ('Ali', 85), ('Sara', 92)]
```

For highest marks first:

3DWEBSOFT FOUNDATION

```
result = sorted(
    students,
    key=lambda student: student[1],
    reverse=True
)

print(result)
```

Output:

```
[('Sara', 92), ('Ali', 85), ('John', 78)
```

11. Small Data-Processing Program

Program 1: Process Student Marks

Suppose students have marks:

```
marks = [45, 67, 82, 91, 56, 74]
```

Find marks greater than or equal to 60:

```
marks = [45, 67, 82, 91, 56, 74]

passed = list(filter(lambda mark: mark >= 60, marks))

print("Passed Marks:", passed)
```

Output:

```
Passed Marks: [67, 82, 91, 74]
```

3DWEBSOFT FOUNDATION

12. Program 2: Increase Marks

Add 5 grace marks to every student:

```
marks = [45, 67, 82, 91, 56]

new_marks = list(map(lambda mark: mark + 5, marks))

print(new_marks)
```

Output:

```
[50, 72, 87, 96, 61]
```

13. Program 3: Convert Names to Uppercase

```
names = ["ali", "sara", "john", "ravi"]

upper_names = list(map(lambda name: name.upper(), names))

print(upper_names)
```

Output:

```
['ALI', 'SARA', 'JOHN', 'RAVI']
```

14. Program 4: Find Numbers Greater Than 50

```
numbers = [20, 65, 45, 80, 30, 90]
```

3DWEBSOFT FOUNDATION

```
result = list(filter(lambda x: x > 50, numbers))


print(result)
```

Output:

```
[65, 80, 90]
```

15. Program 5: Calculate Prices with Discount

Suppose products cost:

```
prices = [1000, 2000, 1500, 3000]
```

Give a 10% discount:

```
prices = [1000, 2000, 1500, 3000]


discounted = list(map(lambda price: price * 0.90, prices))


print(discounted)
```

Output:

```
[900.0, 1800.0, 1350.0, 2700.0]
```

3DWEBSOFT FOUNDATION

16. Mini Project: Student Marks Processor

Problem

Create a program that:

1. Stores student marks.

2. Adds 5 grace marks.
3. Finds students who passed.
4. Sorts the marks.

Practical Code

```
marks = [45, 67, 82, 38, 91, 56, 74]


# Add 5 marks
updated_marks = list(map(lambda x: x + 5, marks))


# Find passed students
passed = list(filter(lambda x: x >= 40, updated_marks))


# Sort marks
sorted_marks = sorted(updated_marks, reverse=True)


print("Original Marks:", marks)
print("Updated Marks:", updated_marks)
print("Passed Marks:", passed)
print("Sorted Marks:", sorted_marks)
```

Expected Output

```
Original Marks: [45, 67, 82, 38, 91, 56, 74]


Updated Marks: [50, 72, 87, 43, 96, 61, 79]
```

SUWEBSOFT FOUNDATION

Passed Marks: [50, 72, 87, 43, 96, 61, 79]

Sorted Marks: [96, 87, 79, 72, 61, 50, 43]

17. Real-World Applications

Lambda functions can be useful for small data-processing tasks such as:

Student Data

Marks → Filter → Passed Students

Shopping Data

Price → Discount → Final Price

Employee Data

Salary → Filter → High Salary Employees

Product Data

Products → Sort → Cheapest/Most Expensive

Data Analysis

Raw Data → Transform → Filter → Sort

18. Important Points to Remember

Lambda functions are **small anonymous functions**.

Syntax:

lambda arguments: expression

Lambda normally contains **one expression**.

3DWEBSOFT FOUNDATION

map() → transforms every item.

filter() → selects items based on a condition.

sorted() → sorts data and can use lambda as a key.

Easy Memory Trick

MAP → CHANGE

FILTER → SELECT

SORTED → ARRANGE

19. Common Mistakes

Mistake 1

lambda x, y:

x + y

Incorrect because a lambda cannot be written like a normal multi-line function.

Correct:

lambda x, y: x + y

Mistake 2

Using lambda for a complicated program.

Avoid:

Very complicated logic inside lambda

For complex logic, use a normal def function.

20. Student Practice Exercises

3DWEBSOFT FOUNDATION

Exercise 1

Create a lambda function to find the square of a number.

Exercise 2

Create a lambda function to find the cube of a number.

Exercise 3

Use map() to double every number:

numbers = [2, 4, 6, 8, 10]

Exercise 4

Use filter() to find odd numbers:

numbers = [1, 2, 3, 4, 5, 6, 7]

Exercise 5

Use lambda and sorted() to sort:

marks = [78, 45, 92, 66, 88]

Exercise 6 – Challenge

Given:

prices = [500, 1200, 750, 2000, 300]

Use lambda and filter() to find products costing more than ₹700.

3DWEBSOFT FOUNDATION

Quick Quiz

1. Which keyword creates a lambda function?

A) def
B) lambda

C) function
D) anonymous

Answer: B

2. What does map() generally do?

A) Deletes data
B) Sorts data
C) Applies an operation to each item
D) Stops a program

Answer: C

3. What does filter() do?

A) Selects items according to a condition
B) Adds items
C) Sorts items
D) Prints items

Answer: A

4. Write a lambda function to add two numbers.

lambda a, b: a + b

5. What is the output?

square = lambda x: x * x

print(square(6))

Answer:

36

22. Classroom Activity

3DWEBSOFT FOUNDATION

Ask students to create a **"Mini Marks Analyzer"**.

They should enter:

Student marks

Then use:

- map() → add grace marks
- filter() → find passing marks
- sorted() → arrange marks
- lambda → perform the operations

This gives students practical experience with **real data processing**, rather than learning lambda functions only as syntax.

3DWEBSOFT FOUNDATION